Arquitectura de Observabilidad con OpenTelemetry

Instrumentación Python, OTel Collector y análisis de overhead

Entregables: código de instrumentación OTel SDK, configuración del OTel Collector y reporte técnico.

Alcance: dos microservicios FastAPI — *orders-service* y *payments-service* — con PostgreSQL, trazas distribuidas, métricas y logs estructurados.

Resumen ejecutivo

La solución separa la generación de telemetría de los backends de observabilidad. Cada servicio Python utiliza OpenTelemetry SDK e instrumentación automática para FastAPI, HTTPX y SQLAlchemy, complementada con spans manuales en operaciones de negocio críticas. Los tres tipos de señal se envían mediante OTLP al OpenTelemetry Collector.

El Collector actúa como gateway de observabilidad: recibe OTLP por gRPC y HTTP, aplica control de memoria, enriquecimiento de recursos y batching, y finalmente enruta trazas hacia Jaeger, métricas hacia un endpoint Prometheus y logs hacia CloudWatch Logs. Esta separación reduce el acoplamiento del código de aplicación con el proveedor de observabilidad y facilita una futura sustitución de backend.

Decisión arquitectónica principal

SDK → OTLP → Collector → backends. No se implementan exporters de Jaeger, Prometheus o CloudWatch directamente dentro de los servicios. El Collector concentra políticas de procesamiento, resiliencia y routing.

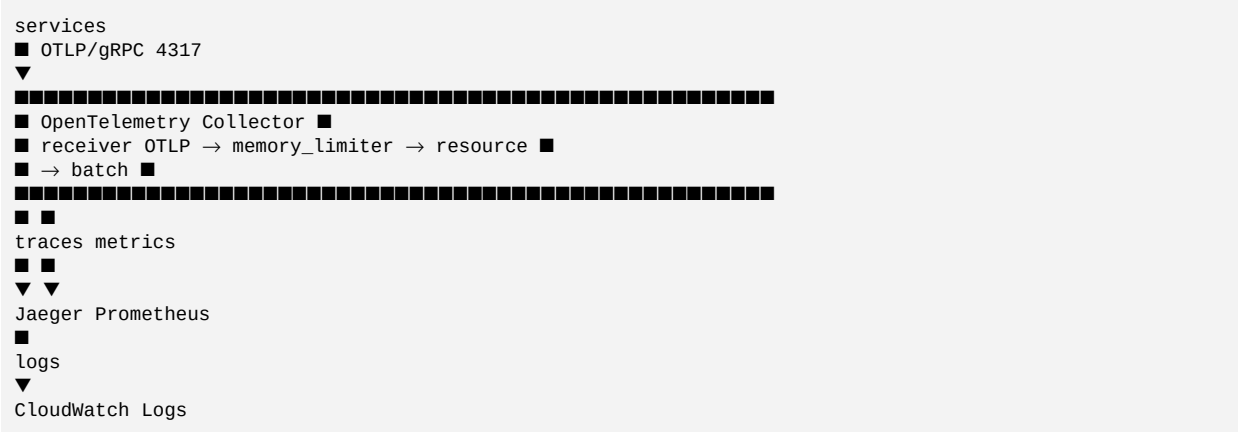
Fecha: 05 de octubre de 2026

1. Arquitectura propuesta

El flujo de una operación comienza en *orders-service*. La creación de una orden genera un span de negocio y una operación SQL instrumentada automáticamente. Posteriormente el servicio llama a *payments-service* usando HTTPX; el contexto W3C se propaga y permite observar la transacción como una sola traza distribuida.

Cliente	orders-service	payments-service	PostgreSQL
HTTP	FastAPI + OTel SDK	FastAPI + OTel SDK	DB spans
	OTLP	OTLP	
	→ Collector	→ Collector	

Capa de observabilidad



La topología corresponde al patrón **gateway Collector**, que centraliza recepción, procesamiento y exportación de señales.

Para el laboratorio local se usa una variante del pipeline de logs con exporter *debug*, evitando depender de credenciales AWS. La configuración de entrega conserva el exporter *awscloudwatchlogs*.

Componentes

Componente	Responsabilidad
FastAPI	API HTTP y puntos de entrada
HTTPX instrumentation	Propagación de contexto entre servicios
SQLAlchemy instrumentation	Spans de operaciones DB
Custom spans	Visibilidad de lógica de negocio crítica
OTel Collector	Procesamiento y routing
Jaeger	Visualización de trazas
Prometheus	Scraping y consulta de métricas
CloudWatch Logs	Destino de logs en AWS

2. Fase 1 — Instrumentación con OTel SDK

La instrumentación combina auto-instrumentación para infraestructura técnica con instrumentación manual para eventos que representan decisiones de negocio. Esta división evita llenar el código con spans redundantes y conserva una señal clara de las operaciones importantes.

2.1 Instrumentación automática

FastAPI genera spans de servidor HTTP. HTTPX genera spans de cliente y permite propagar el contexto W3C hacia el segundo servicio. SQLAlchemy genera spans asociados a la actividad de base de datos. El resultado es una cadena de trazabilidad desde la petición inicial hasta las operaciones SQL.

```
FastAPIInstrumentor.instrument_app(app, tracer_provider=tracer_provider)
HTTPXClientInstrumentor().instrument(tracer_provider=tracer_provider)
SQLAlchemyInstrumentor().instrument(
engine=engine,
tracer_provider=tracer_provider,
)
```

2.2 Spans manuales de negocio

```
with tracer.start_as_current_span("order.create") as span:
span.set_attribute("business.order_id", order_id)
span.set_attribute("business.amount", request.amount)
# persistencia + llamada a payments-service
```

En el servicio de pagos se crea *payment.authorize*. Los atributos se limitan a información útil para diagnóstico y no incluyen secretos. En un entorno real debe aplicarse una política explícita de privacidad y redacción de datos.

2.3 Métricas

Se definen counters para órdenes creadas y pagos autorizados, además de histogramas de duración. El SDK envía métricas por OTLP al Collector; este las expone mediante el exporter Prometheus en *:8889/metrics*, y Prometheus las scrapea.

```
orders.created
orders.create.duration
payments.authorized
payments.authorize.duration
```

2.4 Logs estructurados

Cada evento se serializa como JSON para stdout y se entrega también como registro OTel por OTLP. El JSON incorpora *trace_id* y *span_id* obtenidos del contexto actual, permitiendo correlacionar logs con la traza distribuida.

```
{
"level": "INFO",
"service": "orders-service",
"message": "order completed",
"trace_id": "4f8c...",
"span_id": "a91b..."
}
```

3. Fase 2 — Configuración del OTel Collector

El Collector recibe los tres tipos de señal por OTLP. Se habilitan ambos protocolos solicitados: gRPC en 4317 y HTTP/protobuf en 4318. Esto permite interoperabilidad con clientes que no utilicen gRPC.

3.1 Receiver

```
receivers:
otlp:
protocols:
grpc:
endpoint: 0.0.0.0:4317
http:
endpoint: 0.0.0.0:4318
```

3.2 Processors

memory_limiter: reduce el riesgo de presión de memoria durante picos. **resource:** agrega atributos comunes como entorno y lenguaje. **batch:** agrupa registros antes de exportarlos, reduciendo llamadas y overhead de red.

```
processors:
memory_limiter:
check_interval: 5s
limit_mib: 256
spike_limit_mib: 64

resource:
attributes:
- key: deployment.environment.name
value: local
action: upsert

batch:
send_batch_size: 512
timeout: 5s
```

3.3 Pipeline de trazas

```
traces:
receivers: [otlp]
processors: [memory_limiter, resource, batch]
exporters: [otlp/jaeger]
```

Jaeger actual soporta ingestión OTLP y la solución utiliza su endpoint OTLP/gRPC. La aplicación no necesita conocer detalles del backend; solamente necesita conocer el endpoint del Collector.

3.4 Pipeline de métricas

```
metrics:
receivers: [otlp]
processors: [memory_limiter, resource, batch]
exporters: [prometheus]
```

El exporter Prometheus del Collector expone un endpoint HTTP para scraping. Prometheus consulta *http://otel-collector:8889/metrics*.

3.5 Pipeline de logs

```
logs:
receivers: [otlp]
processors: [memory_limiter, resource, batch]
exporters: [awscloudwatchlogs]
```

El exporter de AWS CloudWatch Logs pertenece a la distribución contrib del Collector. En producción se recomienda IAM Role/Task Role en lugar de claves estáticas. Para el laboratorio se mantiene una configuración local con exporter debug.

4. Análisis de overhead y estrategia de medición

No es correcto afirmar un porcentaje universal de overhead para OpenTelemetry: depende del lenguaje, framework, cantidad de spans, tamaño de atributos, frecuencia de exportación, sampling, volumen de logs y backend. Por eso el entregable incluye una metodología reproducible para medirlo.

4.1 Fuentes de overhead

Fuente	Impacto esperado	Mitigación
Auto-instrumentación	CPU + memoria por spans	Instrumentar librerías necesarias
Custom spans	CPU/memoria proporcional a spans	Solo lógica crítica
Logs	Serialización + I/O	JSON compacto + batching
Métricas	Memoria + export periódico	Intervalo 10 s + cardinalidad controlada
Exportación OTLP	Red + CPU de serialización	Batch y Collector
Collector	Memoria y CPU de procesamiento	memory_limiter + batch

4.2 Diseño experimental

Escenario A — baseline: servicios sin OTel. **Escenario B — auto:** HTTP + DB. **Escenario C — completo:** HTTP + DB + custom spans + métricas + logs. Cada escenario debe ejecutarse con la misma carga, dataset y duración.

Para cada escenario se registran p50, p95 y p99 de latencia, throughput, CPU, memoria RSS, errores HTTP, errores de exportación y tráfico de red. Se recomienda repetir cada prueba varias veces y descartar el calentamiento inicial.

4.3 Hipótesis

La configuración batch y el export asíncrono deben reducir el impacto directo sobre la petición frente a una exportación síncrona por evento. Los custom spans deben tener impacto marginal cuando se limitan a operaciones críticas. Los logs son el componente potencialmente más costoso si el volumen es alto, por lo que deben evitarse mensajes innecesarios y atributos de alta cardinalidad.

4.4 Criterios de aceptación

Como referencia de ingeniería para el piloto, se puede establecer un presupuesto inicial: aumento de p95 ≤ 5%, CPU adicional ≤ 10%, memoria adicional ≤ 15% y ausencia de degradación sostenida del throughput. Estos valores son **criterios de aceptación del proyecto**, no una afirmación de que OpenTelemetry siempre genere ese overhead.

5. Operación, seguridad, pruebas y conclusiones

5.1 Prueba funcional

```
docker compose up --build

curl -X POST http://localhost:8000/orders \
-H 'Content-Type: application/json' \
-d '{"customer_id":"customer-001","amount":125000,"currency":"COP"}'
```

La llamada crea una orden, invoca el servicio de pagos y persiste ambos eventos. En Jaeger debe aparecer una traza con spans HTTP, SQL y los dos spans de negocio. En Prometheus deben aparecer las métricas creadas por los servicios.

5.2 Seguridad

Los atributos de spans y logs deben diseñarse bajo mínimo privilegio de datos. No se deben registrar contraseñas, tokens, credenciales, números completos de tarjetas ni secretos. Fuera de una red confiable, OTLP debe utilizar TLS. En AWS, el Collector debe utilizar roles IAM asociados al workload siempre que sea posible.

5.3 Escalabilidad

La arquitectura puede evolucionar de Docker Compose a Kubernetes, ECS/Fargate u otra plataforma sin modificar el contrato de instrumentación. El patrón Collector gateway permite introducir más instancias del Collector, balanceo y separación de pipelines cuando el volumen crezca.

5.4 Decisiones y trade-offs

Se eligió Python/FastAPI por velocidad de desarrollo y claridad del ejemplo. PostgreSQL representa una dependencia DB real para demostrar auto-instrumentación. Jaeger y Prometheus son backends simples para validar trazas y métricas. CloudWatch se selecciona como destino de logs por alineación con un escenario AWS. La principal concesión es que el exporter CloudWatch pertenece a Collector Contrib y requiere configuración AWS en el entorno de entrega.

Conclusión

La solución cumple las dos fases solicitadas y establece una arquitectura extensible: **instrumentación en el servicio, transporte OTLP, procesamiento centralizado y backends intercambiables**. La separación permite cambiar Jaeger, Prometheus o CloudWatch sin rediseñar la lógica de negocio y proporciona una base adecuada para medir de forma objetiva el overhead de observabilidad.

Referencias técnicas

1. OpenTelemetry — Python instrumentation and SDK. https://opentelemetry.io/docs/languages/python/instrumentation/

2. OpenTelemetry — OTLP exporter configuration. https://opentelemetry.io/docs/languages/sdk-configuration/otlp-exporter/

3. OpenTelemetry — Collector. https://opentelemetry.io/docs/collector/

4. OpenTelemetry — Collector deployment patterns. https://opentelemetry.io/docs/collector/deploy/

5. OpenTelemetry Collector Contrib — AWS CloudWatch Logs exporter. https://github.com/open-telemetry/opentelemetry-collector-contrib/tree/main/exporter/awscloudwatchlogsexporter